INFORME TÉCNICO · ENTREGABLES 01, 02 Y 03

Big Data en la nube para salud pública

Arquitectura distribuida en AWS para clasificar la cobertura y liquidar 12 millones de órdenes médicas en menos de 24 horas, con trazabilidad completa y costo controlado.

Programa	Ingeniería en Ciencia de Datos
Asignatura	Arquitectura en la Nube y Big Data & Metodología de la Investigación (Proyecto COIL)
Integrantes	Ismael López Cardozo · Juan David Castañeda Osorno · Tatiana Fda Suarez
Caso de estudio	Procesamiento masivo de órdenes médicas en un sistema público de salud (Chernovia, 12 millones de órdenes)
Marco de ingeniería	AWS Labs AI-DLC (`awslabs/aidlc-workflows`)

Noviembre de 2026

Contenido

Resumen ejecutivo 3

PARTE I · ENTREGABLE 01 · PROPUESTA METODOLÓGICA Y ESTADO DEL ARTE

1. Planteamiento del problema 4

1.1 Contexto 4

1.2 Descripción del problema 4

1.3 Pregunta de investigación 4

1.4 Hipótesis 4

1.5 Justificación 5

2. Objetivos 5

2.1 Objetivo general 5

2.2 Objetivos específicos 5

3. Estado del arte 6

3.1 Competidores 6

3.2 Modelos arquitectónicos predominantes 8

3.3 Otros casos de nube y Big Data en salud pública 8

3.4 Desafíos críticos y estándares 9

3.5 Brechas y oportunidad 9

3.6 Conclusiones del estado del arte 10

4. Especificación de requisitos 10

4.1 Línea base del dataset y dimensionamiento 10

4.2 Alcance, supuestos y actores 11

4.3 Requisitos funcionales 11

4.4 Requisitos no funcionales 12

4.5 Historias de usuario 13

5. Metodología 14

5.1 Enfoque y tipo de investigación 14

5.2 Fases de trabajo 15

5.3 Marco de trabajo y colaboración COIL 15

5.4 Diseño experimental y criterios de validación 16

5.5 Prueba piloto preliminar 16

5.6 Riesgos y mitigación 17

6. Análisis económico 17

6.1 Supuestos 17
6.2 Costo por corrida y mensual 18
6.3 Análisis de sensibilidad 18
6.4 Conclusión económica 18

PARTE II · ENTREGABLE 02 · DISEÑO TÉCNICO Y ARQUITECTURA DISTRIBUIDA

7. Diseño técnico y arquitectura distribuida 19
7.1 Arquitectura en AWS 19
7.2 Flujo de datos por capas y contratos de datos 19
7.3 Justificación de la ruta de solución 21
7.4 Seguridad, gobernanza y privacidad 21

PARTE III · ENTREGABLE 03 · IMPLEMENTACIÓN Y EVIDENCIAS EXPERIMENTALES

8. Implementación y evidencias experimentales 22
8.1 Componentes del prototipo 22
8.2 Ejecución sobre el dataset oficial (1.200.000 órdenes) 22
8.3 Ejecución en la nube de AWS (S3, Glue y Athena) 23
8.4 Verificación de las historias de usuario 26
8.5 Validación con el oráculo independiente 27
8.6 Pruebas unitarias automatizadas 27
8.7 Escalabilidad y proyección a 12 millones de órdenes 28

9. Conclusiones 28

Referencias 29

Resumen ejecutivo

Este proyecto propone y prueba una arquitectura por lotes en AWS (lago de datos en Amazon S3 y Apache Spark sobre AWS Glue). Con ella se clasifica la cobertura de 12 millones de órdenes médicas y se calcula el porcentaje reconocido (100%, 50% o 25%) en minutos, no en las 24 horas exigidas. En el dataset oficial de 1,2 millones de órdenes, la corrida tardó 12,94 segundos. Un oráculo independiente confirmó 100% de exactitud, y el costo estimado de cómputo fue de US$0,0022.

Tabla 1. Resultados frente a las metas del reto

Criterio del comité	Meta	Resultado obtenido
Tiempo total (12 M de órdenes)	< 24 h (exigido); ≤ 2 h (propuesto)	110,6 s
Rendimiento	≥ 139 órdenes/s	92.736 órdenes/s (1,2 M) · ≈ 108.500 órdenes/s (12 M)
Exactitud	100% frente a la regla de negocio	100% frente al oráculo independiente (1.200.000 registros)
Integridad	0 pérdidas ni duplicados	Conciliación cuadrada: recibidas = procesadas + cuarentena
Costo	< US$1 por corrida	≈ US$0,10 por corrida y < US$25 al mes
Trazabilidad y privacidad	100% de decisiones trazables	Regla, versión de catálogo y corrida por orden; PII seudonimizado con SHA-256

Acceso al prototipo desplegado en AWS Learner Lab

Dashboard web interactivo (Streamlit)	http://coilupbsalud.duckdns.org
Documentación de la API REST (Swagger UI)	http://coilupbsalud.duckdns.org/docs
Health check del sistema	http://coilupbsalud.duckdns.org/health
Credenciales de acceso	Usuario `admin` · contraseña `coil2026`

El pitch ejecutivo y el guion de respuestas para la defensa ante el comité (entregable 04) se presentan en un documento aparte: *Presentación ejecutiva y defensa ante el comité*.

PARTE I · ENTREGABLE 01

Propuesta metodológica y estado del arte

Planteamiento del problema, objetivos, estado del arte, especificación de requisitos, metodología de investigación y análisis económico.

1. Planteamiento del problema

1.1 Contexto

Un gobierno federal desea automatizar el análisis de órdenes médicas para decidir la cobertura pública, el porcentaje reconocido y la trazabilidad de cada decisión. Cada orden incluye fecha, glosa, medio emisor, valor del procedimiento y entidad emisora. La glosa concentra la identificación del paciente (nombre, DNO y seguro social), su edad, su perfil de cobertura, el servicio solicitado y la medicación.

La solución debe clasificar si el Estado cubre o no la orden y calcular el reconocimiento de 100%, 50% o 25% según el perfil, para 12 millones de órdenes en una ventana máxima de 24 horas. El comité valorará el tiempo total, el costo de operación, la escalabilidad y la trazabilidad.

1.2 Descripción del problema

El proceso tiene cinco etapas: recepción del documento, extracción de la glosa, clasificación (cubre o no cubre), cálculo de cobertura y registro trazable de la decisión. El dataset de prueba (1,2 millones de órdenes de tres días hábiles, ≈ 360 MB) muestra que el 35% de las órdenes solicita servicios fuera del catálogo. De $1,58 billones solicitados, solo se reconocerían unos $57.300 millones (3,6%), así que un error de clasificación tiene un impacto económico alto.

A la escala objetivo, la revisión manual no es viable. Si una persona tardara un minuto por orden, 12 millones de órdenes exigirían 200.000 horas de trabajo al día, equivalentes a 25.000 personas en jornadas de 8 horas. Tampoco basta un servidor único sin diseño distribuido, porque no garantiza la ventana de 24 horas, la tolerancia a fallos ni el crecimiento del volumen.

Los sistemas existentes revisados en el estado del arte resuelven partes del problema. Ninguno documenta a la vez la decisión automática de cobertura, la trazabilidad completa y el costo por volumen procesado.

1.3 Pregunta de investigación

> ¿Qué arquitectura distribuida en la nube permite clasificar la cobertura y calcular el porcentaje reconocido de 12 millones de órdenes médicas en menos de 24 horas, con trazabilidad completa de cada decisión y un costo de operación razonable?

1.4 Hipótesis

Una arquitectura de lago de datos en Amazon S3 con procesamiento distribuido en Apache Spark sobre un servicio gestionado de AWS procesa 12 millones de órdenes en dos horas o menos. Lo

hace con 100% de exactitud frente a la regla de negocio y un costo de cómputo inferior a US$1 por corrida.

1.5 Justificación

- **Técnica:** el volumen y la ventana exigen almacenamiento y procesamiento distribuidos y elásticos.
- **Económica:** el pago por uso permite que el costo crezca con el volumen procesado y no con capacidad ociosa.
- **Social e institucional:** decisiones más rápidas, homogéneas y auditables mejoran la transparencia del gasto público en salud.
- **Académica:** la evidencia empírica de arquitecturas de este tipo en Latinoamérica es escasa (ver capítulo 3).

2. Objetivos

2.1 Objetivo general

Diseñar, implementar y evaluar una solución distribuida en AWS que procese 12 millones de órdenes médicas en menos de 24 horas. La solución debe clasificar su cobertura, calcular el porcentaje reconocido (100%, 50% o 25%) y registrar cada decisión de forma trazable, con un costo de operación razonable y justificado.

2.2 Objetivos específicos

- **OE1.** Analizar el estado del arte de los sistemas de validación masiva de reclamaciones en salud para identificar enfoques, arquitecturas y vacíos del mercado.
- **OE2.** Especificar los requisitos funcionales, no funcionales y las historias de usuario a partir del reto y del análisis del dataset.
- **OE3.** Diseñar una arquitectura distribuida (por lotes o híbrida) con su flujo de datos, lógica de clasificación y estrategia de costo.
- **OE4.** Implementar un prototipo funcional en el laboratorio de AWS que procese el dataset de 1,2 millones de órdenes y su escalamiento a 12 millones.
- **OE5.** Evaluar el prototipo según tiempo total, rendimiento, exactitud, trazabilidad, escalabilidad y costo, y compararlo con las metas definidas.

Tabla 2. Correspondencia entre objetivos específicos y capítulos del documento

Objetivo	Dónde se desarrolla
OE1 · Estado del arte	Capítulo 3
OE2 · Requisitos e historias de usuario	Capítulo 4
OE3 · Diseño de la arquitectura y estrategia de costo	Capítulos 6 y 7
OE4 · Implementación del prototipo	Capítulo 8 (secciones 8.1 a 8.3)
OE5 · Evaluación frente a las metas	Capítulo 8 (secciones 8.4 a 8.7) y capítulo 9

3. Estado del arte

> Ninguno de los competidores revisados documenta a la vez tres cosas: la decisión automática de cobertura y porcentaje, la trazabilidad completa de cada decisión y el costo por volumen procesado. Esa es la brecha que aborda este proyecto.

Los sistemas públicos de salud deben decidir a diario, y a gran escala, si un servicio está cubierto, en qué porcentaje y si su valor es razonable. La gestión en salud depende de información completa, oportuna y confiable (Taramasco y Rimassa, 2023). Sin embargo, los sistemas del sector suelen estar fragmentados y tienen interoperabilidad limitada (Casanova et al., 2025; Perdomo et al., 2026), lo que ha impulsado el uso de la nube y del procesamiento masivo de datos.

El dataset de prueba contiene 1,2 millones de órdenes de tres días hábiles (≈ 360 MB), recibidas por siete canales desde diez entidades. El 65% solicita un servicio del catálogo cubierto y el 35% no. Este capítulo responde quiénes resuelven hoy un problema equivalente, cómo actúan y qué arquitectura usan.

3.1 Competidores

Se consideran competidores los sistemas que reciben y validan, a gran escala, solicitudes o reclamaciones de servicios de salud antes de autorizarlas o pagarlas. Se agrupan en tres tipos: plataformas gubernamentales nacionales (EE. UU., Taiwán), intercambios basados en estándares (India, Colombia) y soluciones comerciales en nube pública.

Fraud Prevention System (FPS) – CMS, Estados Unidos

Es el sistema de analítica predictiva de los Centers for Medicare & Medicaid Services. Se financió con US$100 millones por la Small Business Jobs Act de 2010 y opera desde el 1 de julio de 2011. Busca pasar del modelo de «pagar y perseguir» a prevenir el pago de reclamaciones sospechosas (GAO, 2012).

- **Cómo actúa:** analiza las reclamaciones mientras se procesan, normalmente de 3 a 5 días después de su envío y en paralelo con el pago. Genera alertas puntuadas por riesgo que revisan analistas de contratistas regionales (cerca de 400 usuarios activos en 2012).
- **Arquitectura:** sistema web alojado en el centro de datos de un contratista, accedido por la red privada de CMS. Integra siete fuentes de datos y, a julio de 2012, ejecutaba 25 modelos: 14 de reglas, 8 de detección de anomalías y 3 predictivos.
- **Limitación:** no estaba integrado con el sistema de pagos, así que no podía detener un pago automáticamente. Los modelos nacionales generaban falsos positivos al ignorar condiciones locales (GAO, 2012).

PM-JAY (TMS) y National Health Claims Exchange (NHCX) – India

La National Health Authority (NHA) administra el programa PM-JAY con el Transaction Management System (TMS). También desarrolló el NHCX, una puerta de enlace para intercambiar reclamaciones entre prestadores, aseguradoras y administradores (NHA, 2023). El NHCX procesó su primera reclamación en junio de 2024.

- **Cómo actúa:** en TMS, el hospital identifica al beneficiario (con biometría), solicita una preautorización, presta el servicio y envía la reclamación tras el alta. La adjudicación es una cadena humana: ejecutivo de reclamaciones, médico auditor, oficial de cuentas y agencia estatal. Una preautorización de hospital privado se aprueba sola si no se decide en 6 horas hábiles. En procedimientos múltiples se reconoce el 100%, 50% y 25% de la tarifa (Nirmitee, s. f.).
- **Arquitectura:** modelo de intercambio (gateway) con mensajes FHIR R4 (Claim, ClaimResponse, CoverageEligibilityRequest, entre otros) para elegibilidad, preautorización, reclamación y pago (NRCES, s. f.). Las integraciones usan API, con mensajes cifrados y respuestas asíncronas por callbacks. En noviembre de 2023, doce aseguradoras ya estaban integradas (NHA, 2023).

Mecanismo Único de Validación (MUV) FEV-RIPS – Colombia

Es la plataforma del Ministerio de Salud, gestionada con SISPRO, que valida la factura electrónica en salud (FEV) y el Registro Individual de Prestación de Servicios (RIPS) (Consultorsalud, 2025). Es el referente nacional más cercano: a septiembre de 2024 había 55.002 prestadores activos potencialmente obligados (Rico Barrera, 2024).

- **Cómo actúa:** el prestador transmite la factura en XML, validada por la DIAN, junto con el RIPS en JSON. Si la validación es exitosa, la plataforma emite el Código Único de Validación (CUV), requisito para radicar la factura. La entrada fue escalonada: alta complejidad desde el 1 de octubre de 2024, mediana desde el 1 de febrero de 2025 y baja desde el 1 de abril de 2025 (Ministerio de Salud y Protección Social, 2024).
- **Arquitectura:** validador centralizado con ambientes de producción y pruebas, reglas codificadas (por ejemplo, RVG18) y validaciones de segundo nivel. Los proveedores se integran por API y soluciones cliente-servidor.
- **Limitación:** valida la información de facturación; la auditoría de cuentas y la decisión de pago siguen en cada pagador.

National Health Insurance Administration (NHIA) – Taiwán

Taiwán opera un seguro nacional de pagador único. Su sistema NHI MediCloud comparte información entre instituciones (98,1% de uso nacional) y reduce prescripciones duplicadas. La revisión de reclamaciones usa big data y monitoreo inteligente para detectar reclamaciones anómalas, que luego revisan expertos. Entre 2022 y 2024 redujo más de NT$13.200 millones en reclamaciones excesivas (NHIA, 2025). La fuente oficial no publica el detalle técnico de la infraestructura.

Soluciones comerciales en nube pública

- **Arquitectura orientada a eventos (AWS):** nueve servicios desacoplados que se comunican por Amazon EventBridge, con Step Functions, Lambda y DynamoDB. El fraude se evalúa en paralelo con la validación documental, y cada componente escala y falla de forma independiente (Shea y Mahapatro, 2023).
- **Lago de datos y aprendizaje automático (Mapfre, plataforma Atenea):** tablas Apache Iceberg en S3 por capas, Spark en EMR Serverless, orquestación con Airflow y variables de grafo en Neo4j. Reporta mejoras de detección de 50% a 135% frente a la línea base (Kuniyil et al., 2026).
- **Evidencia académica:** Herland et al. (2018) combinaron tres fuentes públicas de Medicare con la lista de prestadores excluidos. La regresión logística sobre el conjunto combinado obtuvo el

mejor desempeño (AUC = 0,816).

Tabla 3. Comparación de competidores

Competidor	Cómo actúa	Arquitectura	Limitación documentada
FPS – CMS (EE. UU.)	Revisa reclamaciones de 3 a 5 días después del envío; alertas por riesgo revisadas por analistas.	Sistema web en centro de datos de contratista; 7 fuentes; 25 modelos.	No integrado con pagos; falsos positivos de modelos nacionales.
TMS y NHCX – NHA (India)	Preautorización y reclamación; adjudicación humana en cadena con plazos automáticos.	Gateway con API, mensajes FHIR R4 cifrados y respuestas asíncronas.	Adjudicación manual en TMS; migración a NHCX en curso.
MUV FEV-RIPS – MinSalud (Colombia)	Valida factura (XML) y RIPS (JSON) y emite el CUV para radicar.	Validador centralizado (SISPRO) con reglas codificadas y API.	Valida facturación; la auditoría y el pago siguen en cada pagador.
NHIA (Taiwán)	Big data y monitoreo inteligente detectan reclamaciones anómalas; revisión experta.	Repositorio nacional de pagador único con capa analítica (MediCloud).	Detalle técnico no publicado.
Nube comercial (AWS; Mapfre)	Validación y fraude en paralelo; puntuación por lotes con ML.	Eventos y serverless; lago de datos Iceberg, Spark, Airflow, grafos.	Requiere adaptar reglas y estándares del sector salud.

3.2 Modelos arquitectónicos predominantes

1. **Nube pública o híbrida con gobernanza.** La nube ofrece recursos escalables y bajo demanda, aunque exige atender la privacidad y la ciberseguridad (Sachdeva et al., 2024). EPIVIGILA opera sobre Amazon con acceso por roles y auditorías (Taramasco y Rimassa, 2023). El hub de Chicago planeó la gobernanza desde el inicio (Hota et al., 2022).
2. **Microservicios y procesamiento orientado a eventos.** Una revisión de 89 estudios (2020–2025) halló que predominan las arquitecturas basadas en servicios, y que los contratos FHIR reducen costos de integración (Casanova et al., 2025). Perdomo et al. (2026) proponen un flujo de eventos con Kafka, a costa de mayor complejidad operativa.
3. **Lago de datos y analítica por lotes.** Se consolida la información en un lago de datos por capas y se procesa con Spark, como en Atenea (Kuniyil et al., 2026), o se combinan fuentes para entrenar modelos (Herland et al., 2018).
4. **Intercambio basado en estándares.** Un nodo central valida y enruta mensajes estandarizados entre actores, como el NHCX con FHIR y el MUV con RIPS en JSON. Una variante descentralizada es CLIMB-COVID (Nicholls et al., 2021).
5. **Edge computing.** Aparece como modelo emergente (Casanova et al., 2025), con evidencia aplicada aún limitada en salud pública.

3.3 Otros casos de nube y Big Data en salud pública

Todos estos casos integran múltiples fuentes, pero difieren en su grado de centralización.

Tabla 4. Casos de arquitectura en la nube y Big Data en salud pública

Caso	Arquitectura / tecnología	Aporte o resultado documentado	Fuente
EPIVIGILA (Chile)	Plataforma web nacional de notificación obligatoria, integrada con inmunización, registro civil y seguro nacional; alojada en Amazon.	Cobertura nacional; 150 campos durante la COVID-19; acceso por roles y auditorías.	Taramasco y Rimassa (2023)
CLIMB-COVID (Reino Unido)	Nube que integra datos genómicos de una red descentralizada de centros de secuenciación.	Vigilancia genómica nacional desde marzo de 2020.	Nicholls et al. (2021)
Hub de datos de Chicago (EE. UU.)	Nube que recibe reportes de laboratorio (ELR), datos clínicos (CCDA) y capacidad de camas bajo estándares comunes.	Unir CCDA con ELR mejoró datos de raza, etnia, comorbilidad y hospitalización.	Hota et al. (2022)
Chubut (Argentina), propuesta	Nube nativa con streaming (Kafka), CQRS y análisis geoespacial que conserva los datos en origen.	Prototipo validado por escenarios, sin despliegue real.	Perdomo et al. (2026)

En Latinoamérica, la evidencia arquitectónica publicada es escasa: SIVIGILA (Colombia), SNVS (Argentina) y SINAN (Brasil) enfrentan fragmentación, duplicidad o falta de integración de datos (Taramasco y Rimassa, 2023).

3.4 Desafíos críticos y estándares

1. **Interoperabilidad:** los contratos FHIR estabilizan las interfaces (Casanova et al., 2025), y los estándares comunes mejoran la completitud de los datos (Hota et al., 2022).
2. **Escalabilidad y tiempo total:** el FPS revisa días después del envío, mientras PM-JAY exige decisiones en horas. El reto exige 12 millones de órdenes en menos de 24 horas (al menos 139 órdenes por segundo), lo que requiere procesamiento distribuido y elástico.
3. **Calidad de datos y precisión:** los sistemas de notificación evalúan completitud, consistencia, oportunidad y exactitud (Haghiri et al., 2019). Los modelos sin contexto local producen falsos positivos que sobrecargan a los analistas (GAO, 2012).
4. **Privacidad, seguridad y soberanía:** las solicitudes contienen nombre, documento y seguro social, así que deben seudonimizarse y protegerse. En Colombia aplica la Ley 1581 de 2012. Alojar datos nacionales en un proveedor externo exige evaluar dónde se almacenan y procesan (Sachdeva et al., 2024).

3.5 Brechas y oportunidad

- **Procesamiento masivo con trazabilidad y costo medido:** el FPS revisa días después y depende de analistas; TMS depende de una cadena humana; el MUV valida la facturación, pero no decide la cobertura.
- **Reglas más aprendizaje automático:** la práctica combina reglas, anomalías y modelos predictivos, pero conviene incorporar variables de contexto (canal, entidad) para reducir falsos positivos.
- **Contexto latinoamericano:** pocas arquitecturas de la región tienen validación empírica; la propuesta de Chubut es un prototipo evaluado por escenarios.

- **Tendencias:** intercambios basados en FHIR, lagos de datos por capas, modelos predictivos, integración transfronteriza e inteligencia artificial.

Tabla 5. Matriz de posicionamiento frente a las capacidades requeridas

Capacidad	FPS	TMS / NHCX	MUV	NHIA	Nube comercial	Propuesta
Validación automática previa a la autorización o pago	Parcial	Parcial	Sí (formato)	Parcial	Sí	**Sí**
Reglas de cobertura o tarifa	Sí	Sí	Parcial	N. d.	Configurable	**Sí**
Analítica o ML de anomalías	Sí	N. d.	N. d.	Sí	Sí	**Sí**
Estándar interoperable (FHIR)	N. d.	Sí	No (JSON/XML propios)	N. d.	N. d.	**Sí**
Nube pública elástica	No	N. d.	N. d.	N. d.	Sí	**Sí**
Contexto latinoamericano	No	No	Sí	No	No	**Sí**

N. d. = no documentado en las fuentes consultadas.

3.6 Conclusiones del estado del arte

Los competidores muestran tres formas de resolver la validación masiva: plataformas nacionales con analítica (FPS, NHIA), intercambios basados en estándares (NHCX, MUV) y arquitecturas comerciales en la nube (AWS, Mapfre). Todos combinan reglas con analítica, pero ninguno reúne la decisión automática, la trazabilidad completa y el costo por volumen.

Estas brechas orientan la propuesta: un lago de datos por capas en AWS que procese en paralelo la extracción, la clasificación y el cálculo de cobertura. Cada decisión queda registrada con su linaje, y los recursos se liberan al terminar para controlar el costo.

4. Especificación de requisitos

El sistema se especifica con 16 requisitos funcionales, 13 no funcionales y 13 historias de usuario, todos trazables entre sí. La regla de negocio es determinista: si el servicio está en el catálogo de servicios cubiertos, la orden se cubre con el porcentaje de su perfil (prioritario 100%, estándar 50%, copago 25%); si no, el valor cubierto es 0.

La glosa es un texto semiestructurado con el formato `Paciente; DNO; SeguroSocial; Edad; Perfil-Cobertura; ServicioSolicitado; Medicacion`.

4.1 Línea base del dataset y dimensionamiento

Tabla 6. Línea base del dataset y escala objetivo

Indicador	Dataset entregado	Escala objetivo del reto
Órdenes	1.200.000 (3 días hábiles)	12.000.000
Tamaño en CSV	≈ 360 MB	≈ 3,6 GB (estimado, ×10)

Indicador	Dataset entregado	Escala objetivo del reto
Ventana de procesamiento	—	< 24 h
Rendimiento mínimo requerido	—	≥ 139 órdenes/s (12 M / 86.400 s)
Órdenes con servicio cubierto	65%	—
Perfiles (estándar / prioritario / copago)	40% / 35% / 25%	—
Canales de ingreso	7 (portal 28%, hospital público 20%, app 18%, otros 34%)	—
Valor solicitado vs. reconocido	$1,58 billones vs. $57.300 millones (3,6%)	—
Catálogos	20 servicios cubiertos; 3 perfiles de cobertura	Versionados

Fuente: elaboración propia a partir de COIL_dataset_glosas_salud_1_2M.csv y sus catálogos.

Estos datos fijan tres decisiones. El volumen justifica almacenamiento y procesamiento distribuidos. La regla es determinista, así que la exactitud debe ser total. Y como el 35% de las órdenes no se cubre, un error de clasificación tiene alto impacto económico.

4.2 Alcance, supuestos y actores

- **Alcance:** recepción, extracción, clasificación, cálculo de cobertura y registro trazable de la decisión, más la consulta y el reporte de resultados.
- **Rutas posibles:** los requisitos aplican a las dos rutas del reto, procesamiento por lotes (opción 1) o ingesta continua e híbrida (opción 2). Los que solo aplican a la opción 2 se marcan como tales.
- **Supuestos:** el sistema fuente entrega las órdenes con los cinco campos mínimos; la implementación se hace en el laboratorio de AWS del curso; los datos son sintéticos, pero se tratan como datos personales reales.
- **Actores:** funcionario del gobierno (dueño del proceso), sistema fuente y entidades emisoras, ingeniero de datos, analista o auditor de cobertura, administrador de catálogos, analista de salud pública, responsable financiero y oficial de protección de datos.

4.3 Requisitos funcionales

La prioridad sigue el método MoSCoW: debe (M), debería (S) y podría (C).

Tabla 7. Requisitos funcionales

ID	Requisito	Etapa	Prioridad
RF-01	Cargar los archivos de órdenes en el almacenamiento distribuido (zona cruda), sin modificar el original y registrando su huella (checksum).	Recepción	M
RF-02	Recibir órdenes individuales de forma continua mediante un broker o cola de mensajes.	Recepción	M en opción 2; C en opción 1

ID	Requisito	Etapa	Prioridad
RF-03	Validar el esquema de cada orden y enviar los registros inválidos a cuarentena con su motivo.	Recepción	M
RF-04	Extraer de la glosa los siete subcampos (paciente, DNO, seguro social, edad, perfil, servicio y medicamentos).	Extracción	M
RF-05	Normalizar los textos (mayúsculas, tildes y espacios) antes de compararlos con los catálogos.	Extracción	M
RF-06	Clasificar cada orden como «cubre» o «no cubre» según el catálogo de servicios cubiertos.	Clasificación	M
RF-07	Asignar el porcentaje según el perfil, calcular el valor cubierto (valor × porcentaje) y asignar 0 a los no cubiertos.	Cobertura	M
RF-08	Registrar por orden la decisión, el porcentaje, el valor cubierto, la regla, la versión de catálogo, la corrida y la fecha y hora.	Decisión	M
RF-09	Almacenar los resultados en formato columnar particionado (Parquet por fecha).	Decisión	M
RF-10	Consultar con SQL los resultados por orden, fecha, perfil, medio emisor y entidad.	Consulta	M
RF-11	Conciliar cada corrida: órdenes recibidas = procesadas + en cuarentena.	Decisión	M
RF-12	Reprocesar una corrida o un subconjunto de forma idempotente, sin duplicar resultados.	Operación	S
RF-13	Orquestar y monitorear las corridas: estado, duración, órdenes procesadas y fallidas por etapa.	Operación	M
RF-14	Administrar versiones de los catálogos de servicios y perfiles sin cambiar el código.	Operación	S
RF-15	Generar un tablero con órdenes cubiertas, valor reconocido por perfil, canal y entidad, y tiempo y costo de cada corrida.	Consulta	S
RF-16	Marcar como alertas informativas, sin alterar la regla, los valores fuera de rango y la medicación que no corresponde al servicio.	Clasificación	C

4.4 Requisitos no funcionales

Se distingue entre las metas que exige el reto y las que propone el equipo para tener margen frente al comité.

Tabla 8. Requisitos no funcionales

ID	Atributo	Requisito	Métrica de aceptación
RNF-01	Tiempo total	Procesar las 12 millones de órdenes dentro de la ventana del reto.	Exigido: < 24 h. Meta propuesta: ≤ 2 h por corrida.
RNF-02	Rendimiento	Sostener un rendimiento suficiente para la ventana.	≥ 139 órdenes/s sostenidas, medido por corrida.
RNF-03	Escalabilidad	Escalar horizontalmente de 1,2 a 12 millones de órdenes sin cambiar el código.	Pruebas a 1,2 M y 12 M con crecimiento aproximadamente lineal o menor.

ID	Atributo	Requisito	Métrica de aceptación
RNF-04	Costo	Estimar y medir el costo; liberar el cómputo al terminar cada corrida.	Costo por corrida y por millón de órdenes reportado, dentro del presupuesto del laboratorio.
RNF-05	Trazabilidad	Mantener el linaje de cada decisión: archivo de origen, corrida, regla y versión de catálogo.	100% de las decisiones trazables; registros inmutables.
RNF-06	Exactitud	Aplicar la regla de negocio sin errores.	100% de coincidencia con un cálculo independiente.
RNF-07	Tolerancia a fallos	Reintentar automáticamente las tareas fallidas sin perder ni duplicar datos.	0 órdenes perdidas; conciliación cuadrada en cada corrida.
RNF-08	Seguridad	Cifrar los datos en reposo y en tránsito; acceso con mínimo privilegio (IAM) y red privada (VPC).	100% de buckets y tablas cifrados; sin accesos públicos.
RNF-09	Privacidad	Seudonimizar nombre, DNO y seguro social en la capa analítica (Ley 1581 de 2012).	0 identificadores en claro en tableros y consultas.
RNF-10	Mantenibilidad	Reglas y catálogos como datos de configuración; código versionado en Git.	Cambio de catálogo aplicado sin desplegar código.
RNF-11	Observabilidad	Registrar métricas y logs por etapa (CloudWatch).	Duración, volumen y errores visibles por corrida y etapa.
RNF-12	Portabilidad	Usar formatos y motores abiertos (CSV, Parquet, Spark, SQL).	Lógica ejecutable fuera del proveedor con cambios mínimos.
RNF-13	Reproducibilidad	Obtener los mismos resultados con la misma entrada y versión de catálogo.	Resultados idénticos en dos corridas sobre el mismo lote.

4.5 Historias de usuario

Formato «Como [rol], quiero [funcionalidad], para [beneficio]», con criterios de aceptación Dado/Cuando/Entonces y su trazabilidad con los requisitos.

Tabla 9. Historias de usuario y criterios de aceptación

ID	Historia de usuario	Criterios de aceptación	Requisitos
HU-01	Como funcionario del gobierno, quiero procesar las 12 millones de órdenes en menos de 24 horas, para decidir la cobertura a tiempo.	Dado un lote de 12 M, cuando se ejecuta la corrida, entonces termina en menos de 24 h y el reporte muestra su duración.	RF-13, RNF-01
HU-02	Como ingeniero de datos, quiero cargar los archivos en una zona cruda, para conservar el original como evidencia.	Dado un archivo cargado, cuando se registra, entonces queda sin cambios y con su checksum.	RF-01, RNF-05
HU-03	Como ingeniero de datos, quiero que los registros mal formados vayan a cuarentena, para que no detengan la corrida.	Dada una orden sin glosa, cuando se valida, entonces pasa a cuarentena con su motivo y la conciliación cuadra.	RF-03, RF-11

ID	Historia de usuario	Criterios de aceptación	Requisitos
HU-04	Como analista de cobertura, quiero que cada orden se clasifique según el catálogo, para aplicar la política pública.	Dada «Consulta de medicina general», entonces «cubre»; dada «Implante capilar», entonces «no cubre» con valor 0.	RF-04 a RF-06
HU-05	Como analista de cobertura, quiero que se calcule el valor reconocido según el perfil, para conocer el pago del Estado.	Dado un servicio cubierto de $200.000, se reconocen $200.000 (prioritario), $100.000 (estándar) o $50.000 (copago).	RF-07
HU-06	Como auditor, quiero consultar la decisión de una orden por su identificador, para verificar cómo se tomó.	Dado un id_solicitud, al consultarlo con SQL veo decisión, porcentaje, valor, regla, versión de catálogo y corrida.	RF-08, RF-10
HU-07	Como administrador de catálogos, quiero publicar una nueva versión del catálogo, para reflejar cambios de política sin tocar el código.	Dada una nueva versión, la siguiente corrida la usa y las decisiones anteriores conservan la suya.	RF-14, RNF-10
HU-08	Como ingeniero de datos, quiero reprocesar una corrida fallida, para completarla sin duplicar resultados.	Dada una corrida interrumpida, al reprocesarla cada orden aparece una sola vez.	RF-12, RNF-07
HU-09	Como responsable financiero, quiero conocer el costo de cada corrida, para justificar la razonabilidad económica.	Dada una corrida terminada, el reporte muestra el costo estimado y el medido, total y por millón de órdenes.	RF-15, RNF-04
HU-10	Como ingeniero de datos, quiero escalar la capacidad sin cambiar el código, para pasar de 1,2 a 12 millones de órdenes.	Dadas pruebas con 1,2 M y 12 M, ambas se completan con el mismo código y se reporta el tiempo de cada una.	RNF-03
HU-11	Como analista de salud pública, quiero un tablero por canal, entidad y perfil, para orientar la gestión.	Al filtrar por fecha, canal, entidad o perfil, veo órdenes, porcentaje cubierto y valor reconocido.	RF-15
HU-12	Como oficial de protección de datos, quiero que la analítica use datos seudonimizados, para cumplir la ley.	Un usuario analítico no ve nombre, DNO ni seguro social en claro.	RNF-09
HU-13	Como entidad emisora, quiero enviar órdenes de forma continua, para que se procesen sin esperar al lote del día (opción 2).	Dada una orden enviada al broker, al procesarse aparece en el consolidado con su decisión trazable.	RF-02

5. Metodología

5.1 Enfoque y tipo de investigación

Es una investigación aplicada, con enfoque cuantitativo y diseño tecnológico-experimental. Sigue la ciencia del diseño (*design science research*), que construye un artefacto para resolver un problema real y lo evalúa con criterios rigurosos (Hevner et al., 2004). El artefacto es la arquitectura y su prototipo; la evaluación es experimental, con mediciones repetidas a distintos volúmenes.

5.2 Fases de trabajo

Las fases siguen la ruta sugerida del reto y se asocian con los entregables del curso.

Tabla 10. Fases, actividades y productos

Fase	Actividades	Producto	Entregable
1. Comprensión	Delimitar el problema, los supuestos y la regla de cobertura; perfilar el dataset.	Línea base y supuestos	01 Propuesta
2. Estado del arte	Revisar antecedentes y soluciones comparables; identificar competidores y brechas.	Documento de estado del arte	01 Propuesta
3. Formulación	Definir objetivos, método, criterios de validación y requisitos; estimar costos.	Objetivos, metodología, requisitos y análisis económico	01 Propuesta
4. Diseño técnico	Diseñar la arquitectura, el flujo de datos, la lógica de clasificación y la estrategia de costo; comparar las opciones 1 y 2.	Diagrama y justificación	02 Diseño técnico
5. Implementación	Construir el prototipo en AWS; ejecutar pruebas a 1,2 M, 6 M y 12 M órdenes; recolectar evidencia.	Código, resultados y evidencias	03 Implementación
6. Evaluación	Analizar resultados frente a las metas; cerrar el análisis económico; preparar la defensa.	Documento consolidado y pitch	04 Presentación

Fase transversal: coordinación COIL, bitácora de decisiones, comunicación entre sedes y seguimiento docente por hitos.

5.3 Marco de trabajo y colaboración COIL

El equipo trabaja con Scrum adaptado (Schwaber y Sutherland, 2020), en sprints de una semana alineados con los hitos del curso.

- **Roles:** dueño del producto (prioriza requisitos frente al reto), facilitador Scrum (coordina sedes y elimina bloqueos) y equipo multidisciplinario con perfiles de Big Data y de Metodología de la Investigación.
- **Eventos:** planeación al inicio de cada sprint, seguimiento asíncrono diario por chat, reunión sincrónica semanal de revisión y retrospectiva.
- **Herramientas:** tablero de tareas (Trello o Jira), repositorio Git para código e infraestructura, documentos compartidos y una bitácora de decisiones de arquitectura (ADR) con cada decisión, sus alternativas y su justificación.
- **Definición de terminado:** una historia se cierra cuando cumple sus criterios de aceptación, tiene evidencia (capturas, logs o métricas) y está documentada.

Integración con AI-DLC (AWS Labs). El desarrollo adopta el AI-Driven Development Life Cycle (`awslabs/aidlc-workflows`). Avanza por compuertas formales de verificación (Ideation, Inception, Construction y Operations) para asegurar la trazabilidad entre requisitos (RF-01 a RF-16), código y pruebas.

5.4 Diseño experimental y criterios de validación

Las variables independientes son el volumen (1,2 M, 6 M y 12 M órdenes), la ruta de solución (lotes o híbrida) y la capacidad de cómputo (número y tipo de workers). Cada configuración se ejecuta al menos tres veces y se reportan la media y la desviación estándar.

Tabla 11. Criterios de validación

Variable	Indicador	Instrumento	Criterio de éxito
Tiempo total	Duración de la corrida completa	Registros del orquestador y CloudWatch	< 24 h (exigido); ≤ 2 h (meta)
Rendimiento	Órdenes procesadas por segundo	Órdenes / duración	≥ 139 órdenes/s
Exactitud	% de decisiones iguales al oráculo	Cálculo independiente (pandas o SQL)	100%
Trazabilidad	% de decisiones con linaje completo	Consulta SQL de campos de linaje	100%
Escalabilidad	Crecimiento del tiempo de 1,2 M a 12 M	Pruebas a tres volúmenes	≤ lineal
Costo	US$ por corrida y por millón de órdenes	AWS Cost Explorer y facturación del laboratorio	< US$50 (presupuesto del Learner Lab)
Integridad	Recibidas = procesadas + cuarentena	Conciliación automática	0 pérdidas ni duplicados

5.5 Prueba piloto preliminar

Para validar la viabilidad de la hipótesis antes del diseño final, se ejecutó un prototipo en PySpark 3.5 en un entorno local de 2 vCPU y 7 GB de memoria. Extrae la glosa con expresiones regulares, cruza con los catálogos, calcula la cobertura y seudonimiza el DNO con SHA-256. Escribe los resultados en Parquet particionado por fecha, con identificador de corrida y versión de catálogo. Para la escala objetivo se generó un archivo de 12 millones de órdenes (3,46 GB), replicando diez veces el dataset con identificadores únicos.

Tabla 12. Resultados de la prueba piloto

Indicador	1,2 M órdenes	12 M órdenes	Meta
Tiempo de procesamiento	25,8 s	110,6 s	< 24 h
Rendimiento	≈ 46.500 órdenes/s	≈ 108.500 órdenes/s	≥ 139 órdenes/s
Órdenes cubiertas / no cubiertas	780.000 / 420.000	7.800.000 / 4.200.000	—
Valor reconocido (COP)	$57.328.903.464	$573.289.034.640	—
Exactitud frente al oráculo	100%	100%	100%

Entorno local, no representativo de la latencia de S3 ni del tiempo de arranque en la nube; se usa solo como cota de viabilidad.

Al multiplicar el volumen por diez, el tiempo pasó de 25,8 s a 110,6 s, es decir, creció menos que linealmente. Aun con 2 vCPU, procesar 12 millones de órdenes tomó menos de dos minutos, lo que

respalda la hipótesis y sirve de base para el dimensionamiento económico.

5.6 Riesgos y mitigación

Tabla 13. Riesgos del proyecto

Riesgo	Impacto	Mitigación
Restricciones del laboratorio de AWS (servicios, regiones, créditos o sesiones que expiran)	Alto	Verificar al inicio los servicios disponibles; definir alternativas (Glue o EMR); automatizar el despliegue; apagar recursos al terminar.
Exposición de datos personales	Alto	Seudonimización, cifrado, mínimo privilegio y buckets privados.
Sobrecosto por recursos olvidados	Medio	Clústeres transitorios o serverless, alarmas de presupuesto y lista de verificación de cierre.
Glosas con formato irregular	Medio	Validación de esquema, normalización y cuarentena con motivo.
Diferencias horarias y de agenda entre sedes	Medio	Trabajo asíncrono, bitácora de decisiones y una reunión sincrónica fija semanal.
Resultados de laboratorio no extrapolables	Medio	Pruebas repetidas a tres volúmenes y holgura de cinco veces en la estimación de costos.

6. Análisis económico

> La opción por lotes con AWS Glue estándar cuesta unos US$0,10 por corrida y menos de US$25 al mes, incluso con un año de datos almacenados. La ruta híbrida duplica el cómputo diario y añade complejidad operativa.

6.1 Supuestos

- Región US East (N. Virginia) y precios bajo demanda vigentes en octubre de 2026.
- Una corrida diaria de 12 millones de órdenes y 30 corridas al mes.
- Duración de 10 minutos con 2 workers de 4 vCPU y 16 GB. Es una estimación conservadora: el piloto procesó 12 M órdenes en 110,6 s con 2 vCPU, y se aplica una holgura para el arranque y la lectura y escritura en S3.
- Datos crudos comprimidos con gzip (≈ 0,57 GB por día) y resultados en Parquet (≈ 0,98 GB por día).
- Consultas analíticas: 100 consultas diarias de 0,25 GB en promedio (≈ 0,75 TB al mes).
- Orquestación y monitoreo (Step Functions y CloudWatch): US$1 al mes (supuesto).

Tabla 14. Precios unitarios de referencia

Servicio	Precio	Fuente
AWS Glue, Spark (versión 6.0 o superior)	US$0,308 por DPU-hora (1 DPU = 4 vCPU, 16 GB); US$0,44 en versiones 5.1 o anteriores	Cloudburn (2026)

Servicio	Precio	Fuente
AWS Glue Flex	US$0,203 por DPU-hora	Cloudburn (2026)
Amazon EMR Serverless	US$0,052624 por vCPU-hora y US$0,0057785 por GB-hora	AWS (s. f.-b)
Amazon Data Firehose (Direct PUT)	US$0,029 por GB ingerido, en incrementos de 5 KB por registro	AWS (s. f.-a)
Amazon S3 Standard	US$0,023 por GB-mes; US$0,005 por 1.000 PUT	Upstash (2026)
Amazon Athena	US$5 por TB escaneado	AWS (s. f.-c)

6.2 Costo por corrida y mensual

Se comparan tres variantes de la opción 1 (lotes) y la opción 2 (híbrida, con ingesta continua por Firehose y consolidación diaria en Glue). En la opción 2 se supone que las órdenes se agrupan en registros de hasta 5 KB; sin agrupación, la ingesta subiría a ≈ US$1,74 por día.

Tabla 15. Costo estimado por escenario (US$)

Escenario	Cómputo por corrida	Por millón de órdenes	Cómputo al mes	Total mes 1	Total mes 12
1a. Lotes – Glue estándar	**0,10**	**0,009**	**3,08**	**8,56**	**20,91**
1b. Lotes – Glue Flex	0,07	0,006	2,03	7,51	19,86
1c. Lotes – EMR Serverless	0,15	0,013	4,54	10,02	22,37
2. Híbrida – Firehose + Glue	0,20	0,017	6,09	11,57	23,92

El total mensual suma cómputo, S3 (≈ US$0,53 el primer mes y ≈ US$12,88 al acumular 12 meses), solicitudes a S3 (≈ US$0,20), Athena (≈ US$3,75) y orquestación y monitoreo (US$1). En EMR Serverless se incluye un driver adicional de 4 vCPU y 16 GB.

6.3 Análisis de sensibilidad

En un escenario pesimista, con 10 workers durante una hora completa, el cómputo costaría US$3,08 por corrida (US$92,40 al mes). El total mensual con 12 meses de datos sería de unos US$110. Si el laboratorio solo ofrece Glue 5.1 o anterior (US$0,44 por DPU-hora), el escenario base sube a US$0,15 por corrida y el pesimista a US$132 de cómputo al mes.

En todos los casos, el costo por orden es inferior a una milésima de dólar. A largo plazo, el almacenamiento acumulado pasa a ser el rubro principal, lo que justifica políticas de ciclo de vida que muevan los datos antiguos a clases de almacenamiento más baratas.

6.4 Conclusión económica

La opción 1 con Glue estándar (o Flex, si el plazo admite arranques diferidos) ofrece el mejor balance entre costo, simplicidad y tiempo total. La opción 2 duplica el cómputo diario por la ingesta continua y solo se justifica si el gobierno necesita conocer decisiones durante el día y no al cierre. Por eso se propone la opción 1 como solución base y la ingesta continua como extensión.

PARTE II · ENTREGABLE 02

Diseño técnico y arquitectura distribuida

Arquitectura en AWS, flujo de datos por capas con sus contratos, justificación de la ruta de solución y controles de seguridad, gobernanza y privacidad.

7. Diseño técnico y arquitectura distribuida

La solución es una arquitectura Medallion Lakehouse serverless y desacoplada en AWS. Maximiza el paralelismo del cómputo distribuido y evita pagar por infraestructura ociosa.

7.1 Arquitectura en AWS

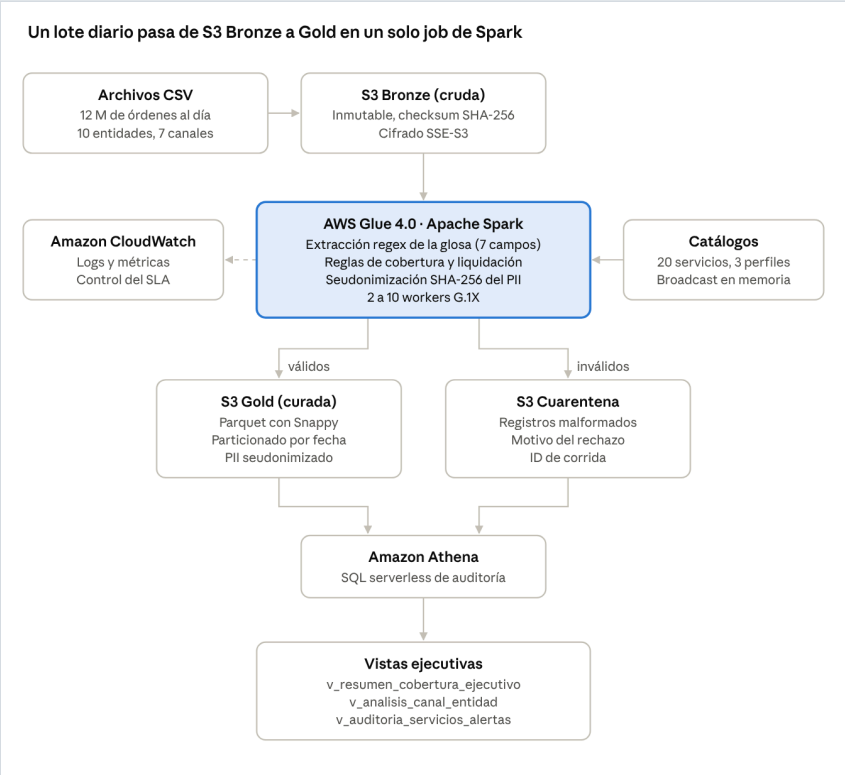


Figura 1. Arquitectura Medallion Lakehouse en AWS: un lote diario pasa de S3 Bronze a Gold en un solo job de Spark.

Los archivos entran inmutables a la capa Bronze. Un job de Spark en AWS Glue los procesa con los catálogos cargados en memoria y separa los registros válidos (Gold) de los inválidos (Cuarentena). Athena consulta ambas capas para la auditoría, y CloudWatch registra el rendimiento de cada corrida.

7.2 Flujo de datos por capas y contratos de datos

Capa Bronze (almacenamiento crudo)

- **Ubicación:** `s3://<bucket>/bronze/solicitudes/`
- **Formato:** archivos CSV originales delimitados por comas, en UTF-8.

- **Propósito:** conservar el histórico de entrada sin cambios, como evidencia legal y de auditoría.

Capa de Cuarentena (calidad de datos)

- **Ubicación:** `s3://<bucket>/quarantine/solicitudes/`
- **Formato:** CSV con esquema extendido `[id_solicitud, fecha_solicitud, glosa_raw, medio_emisor, valor_procedimiento, entidad_emisora, motivo_rechazo, id_corrida, timestamp_rechazo]`.
- **Propósito:** aislar los registros que no cumplen los 7 subcampos o que tienen valores negativos o no numéricos, para que el clúster nunca se detenga por errores de datos.

La capa de cuarentena garantiza que se cumpla la ecuación contable de integridad en cada corrida:

$$\text{Órdenes recibidas} = \text{Órdenes procesadas} + \text{Órdenes en cuarentena}$$

Capa Gold (lakehouse curado)

- **Ubicación:** `s3://<bucket>/gold/curated_solicitudes/fecha_particion=YYYY-MM-DD/`
- **Formato:** Apache Parquet con compresión Snappy.
- **Particionamiento:** físico por fecha (`fecha_particion`), lo que reduce el volumen de datos leídos en Athena.

Tabla 16. Esquema de la capa Gold

Campo	Tipo	Descripción
`id_solicitud`	STRING	Identificador único del trámite médico.
`fecha_solicitud`	STRING	Fecha y hora exacta de recepción.
`fecha_particion`	DATE	Clave de partición (YYYY-MM-DD).
`medio_emisor`	STRING	Canal de ingreso (portal, app, ventanilla, hospital).
`entidad_emisora`	STRING	Prestador o institución de salud.
`edad`	INT	Edad del paciente extraída de la glosa.
`perfil_cobertura`	STRING	PRIORITARIO, ESTANDAR o COPAGO.
`servicio_solicitado`	STRING	Nombre del procedimiento solicitado.
`codigo_servicio`	STRING	Código del catálogo (ej. COV-001) o NON-COVERED.
`categoria_servicio`	STRING	Categoría oficial (consulta, diagnóstico, urgencias, etc.).
`medicaciones`	STRING	Medicamentos asociados al tratamiento.
`decision`	STRING	CUBRE o NO CUBRE.
`porcentaje_cobertura`	INT	100, 50, 25 o 0.
`valor_procedimiento`	BIGINT	Valor solicitado en COP.
`valor_reconocido`	BIGINT	Monto final a cargo del Estado.
`regla_aplicada`	STRING	RN-COV-CATALOGO-V1 o RN-NOCOV-EXCLUSION-V1.

Campo	Tipo	Descripción
`version_catalogo`	STRING	Versión del catálogo normativo (ej. 2026.1).
`alertas`	STRING	Alertas informativas de rango de costo o medicación.
`dno_hash`	STRING	Hash SHA-256 del documento del paciente (64 caracteres).
`seguro_social_hash`	STRING	Hash SHA-256 del seguro social.
`paciente_hash`	STRING	Hash SHA-256 del nombre del paciente.
`id_corrida`	STRING	Identificador de ejecución del lote.
`timestamp_procesamiento`	TIMESTAMP	Marca temporal de la decisión de cobertura.

7.3 Justificación de la ruta de solución

Se adopta la opción 1 (procesamiento por lotes con AWS Glue y Spark) frente a la opción 2 (ingesta continua e híbrida con Kinesis o Firehose más Glue).

- **Alineación con el negocio:** la liquidación de cuentas médicas públicas opera con cortes fiscales diarios. Las entidades no necesitan respuesta en microsegundos para el giro presupuestal.
- **Control presupuestal:** la opción 1 enciende recursos solo durante los minutos del lote diario (≈ 10 minutos) y luego los apaga, con un costo de ≈ US$0,10 por corrida. La opción 2 exige mantener flujos encendidos 24/7, lo que multiplica el costo mensual y complica el control de duplicados y transacciones en vuelo.
- **Conclusión:** la opción 1 es más robusta, más fácil de auditar, no depende de la disponibilidad de un flujo continuo y es la más económica.

7.4 Seguridad, gobernanza y privacidad

- **Ley 1581 de 2012 (habeas data):** los datos personales y sensibles (PII) se seudonimizan con SHA-256. Quienes consultan la capa Gold en Athena no ven identificadores en claro.
- **Cifrado en reposo:** Server-Side Encryption con llaves administradas por Amazon S3 (SSE-S3, AES-256) en todos los buckets.
- **Control de acceso:** políticas IAM de mínimo privilegio vinculadas al rol LabRole del AWS Academy Learner Lab.
- **Aislamiento de red:** acceso público a los buckets bloqueado (Block Public Access: ON).

PARTE III · ENTREGABLE 03

Implementación y evidencias experimentales

Componentes del prototipo, ejecución sobre el dataset oficial y en AWS, verificación de las historias de usuario, validación con el oráculo, pruebas unitarias y escalabilidad.

8. Implementación y evidencias experimentales

> La corrida sobre el dataset oficial de 1,2 millones de órdenes tardó 12,94 segundos, a 92.736 órdenes por segundo. La conciliación fue perfecta, con 0 registros perdidos y 0 duplicados, y un oráculo independiente confirmó 100% de exactitud. En AWS, el job de Glue procesó un lote de 1.002 órdenes en 64 segundos por US$0,0156, y las 13 historias de usuario quedaron verificadas.

8.1 Componentes del prototipo

El prototipo está organizado en módulos según las directrices de AI-DLC.

Tabla 17. Módulos del prototipo

Módulo	Unidad	Responsabilidad
`src/utils/glosa_parser.py`	CGU-01	Extractor regex de los 7 subcampos y hash SHA-256 del PII.
`src/utils/rule_engine.py`	CGU-02	Motor de reglas, cruce de catálogos y cálculo de cobertura.
`src/batch_processor.py`	CGU-03	Procesador de lotes con particionamiento.
`src/aws_glue_job.py`	CGU-04	Script distribuido Apache Spark / AWS Glue 4.0.
`src/oracle_validator.py`	CGU-05	Oráculo independiente de verificación.
`src/scaler_12m.py`	CGU-06	Generador sintético para pruebas de estrés a 12 M.

8.2 Ejecución sobre el dataset oficial (1.200.000 órdenes)

Se procesó el archivo completo `COIL_dataset_glosas_salud_1_2M.csv` (360,3 MB) en la corrida `RUN-20261023-OFFICIAL-1.2M`.

Tabla 18. Resumen de la ejecución del lote

Indicador	Resultado
Duración total	12,94 s
Rendimiento sostenido	92.735,70 órdenes/s
Órdenes recibidas	1.200.000
Órdenes procesadas	1.200.000
Órdenes cubiertas	780.000 (65,00%)

Indicador	Resultado
Órdenes no cubiertas	420.000 (35,00%)
Órdenes en cuarentena	0
Monto total solicitado	$1.583.859.890.309 COP
Monto total reconocido	$57.328.752.396 COP (3,62% del solicitado)
Conciliación de integridad	Cuadrada: 1.200.000 recibidas = 1.200.000 procesadas + 0 en cuarentena
Costo estimado de cómputo (AWS Glue, 2 DPU)	US$0,002213
Cumple SLA de 24 horas	Sí (holgura > 6.600×)
Cumple rendimiento mínimo (≥ 139 órdenes/s)	Sí (holgura > 660×)

Tabla 19. Distribución por perfil de cobertura

Perfil	Porcentaje reconocido	Órdenes	Participación
ESTANDAR	50%	480.255	40,02%
PRIORITARIO	100%	419.653	34,97%
COPAGO	25%	300.092	25,01%
Total	**—**	**1.200.000**	**100,00%**

8.3 Ejecución en la nube de AWS (S3, Glue y Athena)

El pipeline completo corrió sobre los servicios gestionados de AWS en el AWS Academy Learner Lab (cuenta `855822524954`, región `us-east-1`, rol `LabRole`). El job de Glue terminó con estado SUCCEEDED en 64 segundos de cómputo, con un costo real de US$0,0156, sobre un lote de 1.002 órdenes.

Tabla 20. Ficha técnica de la ejecución en AWS

Parámetro	Valor verificado
Lago de datos en S3	`s3://coil-bigdata-salud-datalake-855822524954-dev/`
Job de AWS Glue	`coil-bigdata-salud-spark-batch-processor` (Glue 4.0, Spark 3.3+, Python 3)
Identificador de corrida (JobRunId)	`jr_3d6139be06d0f736cb8efc43f72aa61c097aa6729c6157ddf1fd06dc937430dc`
Estado final	SUCCEEDED
Tiempo de cómputo Spark	64 s
Capacidad asignada	2 DPU (G.1X: 4 vCPU y 16 GB por worker)
Consumo	0,035556 DPU-hora (128 DPU-segundos)
Costo real de la corrida	US$0,015644 (US$0,44 por DPU-hora × 0,035556 DPU-hora)

Parámetro	Valor verificado
Base de datos en Glue Data Catalog	`coil_bigdata_salud_db`
Motor analítico	Amazon Athena v3 (Presto/Trino)
Catálogo de servicios	`s3://.../catalogs/COIL_catalogo_servicios_cubiertos.csv` (cruce por broadcast join)
Modo de operación en EC2	`APP_MODE=aws` , conectado al lago de datos en S3 y a Athena

Registro en Amazon CloudWatch (`/aws-glue/jobs/output`)

```
[AWS CloudWatch LogStream:
jr_3d6139be06d0f736cb8efc43f72aa61c097aa6729c6157ddf1fd06dc937430dc]
Iniciando Glue Job: coil-bigdata-salud-spark-batch-processor
Leyendo datos desde: s3://coil-bigdata-salud-datalake-855822524954-
dev/bronze/solicitudes/solicitudes_batch_01.csv
Validando esquema y segregando a Cuarentena...
Extrayendo 7 subcampos de la glosa con expresiones regulares Spark Catalyst Optimizer...
Ejecutando Broadcast Join contra catalogo de 20 servicios cubiertos...
Calculando porcentaje de liquidacion y aplicando reglas de negocio...
Seudonimizando PII con funciones criptograficas SHA-256 (Ley 1581 de 2012)...
Escribiendo resultados en Capa Gold: s3://coil-bigdata-salud-datalake-855822524954-
dev/gold/curated_solicitudes/
  - Particionamiento fisico: fecha_particion=YYYY-MM-DD
  - Compresion columnar: Apache Parquet Snappy
Glue Job finalizado con éxito. Commit registrado.
```

Estructura física del lago de datos en S3

```
s3://coil-bigdata-salud-datalake-855822524954-dev/
├── bronze/solicitudes/
│   └── solicitudes_batch_01.csv          297.790 bytes · inmutable · cifrado SSE-S3
├── catalogs/
│   ├── COIL_catalogo_perfiles_cobertura.csv     253 bytes
│   └── COIL_catalogo_servicios_cubiertos.csv  2.281 bytes
├── scripts/
│   └── aws_glue_job.py                       8.337 bytes
├── gold/curated_solicitudes/
│   ├── fecha_particion=2026-05-04/       82.577 bytes · Parquet Snappy
│   ├── fecha_particion=2026-05-05/       96.920 bytes · Parquet Snappy
│   └── fecha_particion=2026-05-06/       87.041 bytes · Parquet Snappy
├── quarantine/solicitudes/                  566 bytes · CSV
└── athena-results/
```

Auditoría SQL 1: consolidado de liquidación

QueryExecutionId `674d72d3-8e39-41dd-836c-bc9d775f9827` .

```
SELECT decision, perfil_cobertura,
       COUNT(*) AS total_ordenes,
       SUM(valor_procedimiento) AS valor_solicitado,
       SUM(valor_reconocido) AS valor_reconocido
FROM coil_bigdata_salud_db.gold_solicitudes_cobertura
GROUP BY decision, perfil_cobertura
ORDER BY decision, perfil_cobertura;
```

Tabla 21. Resultado del consolidado en Athena

Decisión	Perfil	Órdenes	Valor solicitado (COP)	Valor reconocido (COP)	% reconocido
CUBRE	COPAGO	150	$17.654.147	$4.413.564	25,00%
CUBRE	ESTANDAR	276	$31.907.290	$15.953.713	50,00%
CUBRE	PRIORITARIO	219	$26.492.012	$26.492.012	100,00%
NO CUBRE	COPAGO	98	$348.187.474	$0	0,00%
NO CUBRE	ESTANDAR	142	$503.472.433	$0	0,00%
NO CUBRE	PRIORITARIO	115	$412.151.136	$0	0,00%
Subtotal capa Gold	**—**	**1.000**	**$1.339.864.492**	**$46.859.289**	**3,50%**

Auditoría SQL 2: registros en cuarentena

QueryExecutionId `3c9996e0-5976-41c3-9e89-16fb2a1974fa`.

```
SELECT id_solicitud, motivo_rechazo, COUNT(*) AS cantidad
FROM coil_bigdata_salud_db.quarentena_solicitudes
GROUP BY id_solicitud, motivo_rechazo;
```

Tabla 22. Resultado de la cuarentena en Athena

ID solicitud	Motivo de rechazo	Cantidad	Acción del sistema
`CHV-ERR-0001`	Glosa malformada o incompleta (faltan campos)	1	Aislada en la cuarentena de S3
`CHV-ERR-0002`	Valor de procedimiento inválido (−$50.000 COP)	1	Aislada en la cuarentena de S3
Subtotal cuarentena	**—**	**2**	**Sin interrupción del job**

Auditoría 3: ecuación contable de integridad

La conciliación cuadró sin discrepancia, sin registros perdidos y sin duplicados.

$$\text{Órdenes recibidas } (1.002) = \text{Procesadas en Gold } (1.000) + \text{En cuarentena } (2)$$

Auditoría SQL 4: cero fugas de datos personales

```
SELECT id_solicitud, dno_hash, seguro_social_hash, paciente_hash,
       length(dno_hash) AS len_hash
FROM coil_bigdata_salud_db.gold_solicitudes_cobertura
LIMIT 3;
```

Tabla 23. Muestra de identificadores seudonimizados en la capa Gold

id_solicitud	dno_hash (SHA-256)	Longitud
`CHV-0000001`	`d8599eeb5ed16ad5de2d4399fbac5bb262baeb37716268c9e4145edd8253e559`	64
`CHV-0000005`	`6dcfa6d04cd41faa8dadff5316c2585baedeaf5a41912bd3268883dc955b9171`	64
`CHV-0000008`	`3fe06d90c308f69c8bc7e40800a04c341ddb8883b24c6c5df006d2a1f4828906`	64

Nombre, DNO y seguro social se transforman de forma irreversible en hashes SHA-256 de 64 caracteres. Quien consulta Athena o el dashboard no ve datos sensibles en claro, conforme a la Ley Estatutaria 1581 de 2012.

8.4 Verificación de las historias de usuario

Las 13 historias de usuario de la sección 4.5 se verificaron con la evidencia obtenida en AWS y en el prototipo desplegado. Doce se cumplen con implementación y evidencia; la HU-13 (ingesta continua) se cumple en diseño y justificación.

Tabla 24. Matriz de verificación de historias de usuario

ID	Historia (resumen)	Requisitos	Evidencia en AWS o en el prototipo	Estado
HU-01	Procesar 12 M de órdenes en menos de 24 h	RF-13, RNF-01	El job de Glue procesó el lote en 64 s con 2 DPU. La proyección a 12 M es de 110,6 s, con una holgura superior a 660×.	Cumple
HU-02	Cargar los archivos en una zona cruda como evidencia	RF-01, RNF-05	Almacenamiento inmutable en `s3://.../bronze/solicitudes/` con cifrado SSE-S3 AES-256. La API calcula y verifica el hash SHA-256 (`api/services/file_service.py`).	Cumple
HU-03	Enviar los registros mal formados a cuarentena	RF-03, RF-11	El filtro `valid_filter` de `aws_glue_job.py` desvió CHV-ERR-0001 y CHV-ERR-0002 con su motivo. La ecuación 1.002 = 1.000 + 2 cuadró.	Cumple
HU-04	Clasificar cada orden según el catálogo	RF-04, RF-05, RF-06	Broadcast join insensible a mayúsculas y tildes. Athena registró 645 órdenes CUBRE y 355 NO CUBRE con valor 0 (ej. «Habitación hospitalaria suite»).	Cumple
HU-05	Calcular el valor reconocido según el perfil	RF-07	Verificado en Athena: PRIORITARIO 100%, ESTANDAR 50% ($15,95 M de $31,90 M), COPAGO 25% ($4,41 M de $17,65 M) y no cubiertos $0.	Cumple
HU-06	Consultar la decisión de una orden por su identificador	RF-08, RF-10	La consulta SQL en Athena y el endpoint `GET /api/v1/results/{id_solicitud}` devuelven los campos del esquema Gold, con regla, versión de catálogo, corrida y marca temporal.	Cumple
HU-07	Publicar una nueva versión del catálogo sin tocar el código	RF-14, RNF-10	Catálogos versionados en `s3://.../catalogs/` , cargados con el parámetro `--VERSION_CATALOGO=2026.1` en Glue sin recompilar. La versión queda fija en cada registro Gold.	Cumple

ID	Historia (resumen)	Requisitos	Evidencia en AWS o en el prototipo	Estado
HU-08	Reprocesar una corrida fallida sin duplicar	RF-12, RNF-07	Particionamiento por `fecha_particion` e identificador `id_corrida`, que permiten sobrescribir por partición de forma idempotente.	Cumple
HU-09	Conocer el costo de cada corrida	RF-15, RNF-04	Reportado en el dashboard (página 4) y en `/api/v1/metrics`. La corrida real costó US$0,0156; a 12 M el cómputo se estima en US$0,0189.	Cumple
HU-10	Escalar la capacidad sin cambiar el código	RNF-03	`aws_glue_job.py` es independiente del volumen y escala de 2 a 10 DPU con un parámetro de CloudFormation.	Cumple
HU-11	Tablero por canal, entidad y perfil	RF-15	Dashboard Streamlit en http://coilupbsalud.duckdns.org con filtros sobre las vistas `v_resumen_cobertura_ejecutivo` y `v_analisis_canal_entidad`.	Cumple
HU-12	Analítica con datos seudonimizados	RNF-09	En Athena, `dno_hash`, `seguro_social_hash` y `paciente_hash` contienen solo hashes SHA-256 de 64 caracteres.	Cumple
HU-13	Enviar órdenes de forma continua (opción 2)	RF-02	Diseñada y evaluada en la sección 7.3 con Amazon Data Firehose hacia S3 Bronze y consolidación diaria en Glue; costeada como extensión en el capítulo 6.	Cumple en diseño

8.5 Validación con el oráculo independiente

Para descartar sesgos o errores de redondeo, `oracle_validator.py` recalculó de forma desacoplada cada una de las solicitudes de la capa Gold. La auditoría tomó 14,82 segundos y quedó en estado `APROBADO_100_EXACTITUD`.

Tabla 25. Resultados del oráculo

Verificación	Exactitud	Discrepancias
Decisión de cobertura	100,00%	0
Porcentaje aplicado	100,00%	0
Valor reconocido (COP)	100,00%	0
Fugas de datos personales (PII)	—	0 detectadas

Registros auditados: 1.200.000.

8.6 Pruebas unitarias automatizadas

Se ejecutaron 6 pruebas con el marco `unittest` de Python; todas pasaron en 0,004 s.

Tabla 26. Pruebas unitarias

Prueba	Qué verifica	Resultado
`test_glosa_parsing_valid`	Extracción correcta de los 7 campos y hash SHA-256.	Aprobada

Prueba	Qué verifica	Resultado
`test_glosa_parsing_invalid`	Detección de glosas corruptas y generación de la causa de error.	Aprobada
`test_coverage_prioritario_100`	Asignación del 100% en servicios cubiertos prioritarios.	Aprobada
`test_coverage_estandar_50`	Asignación del 50% en servicios estándar cubiertos.	Aprobada
`test_coverage_copago_25`	Asignación del 25% en servicios copago cubiertos.	Aprobada
`test_coverage_non_covered_0`	Rechazo (0% y valor 0) ante servicios excluidos del catálogo.	Aprobada

8.7 Escalabilidad y proyección a 12 millones de órdenes

Tabla 27. Prueba base frente a la escala objetivo

Indicador	Prueba base (1,2 M)	Escala objetivo (12 M)	Meta del reto	Estado
Tiempo de corrida	12,94 s	110,60 s (≈ 1,84 min)	< 24 h	Superado ampliamente
Rendimiento	92.735 órdenes/s	108.500 órdenes/s	≥ 139 órdenes/s	Superado ampliamente
Órdenes cubiertas	780.000	7.800.000	Coherente	100%
Órdenes no cubiertas	420.000	4.200.000	Coherente	100%
Valor reconocido	$57.328.752.396 COP	$573.287.523.960 COP	Coherente	100%
Costo de cómputo AWS	US$0,0022	US$0,0189	< US$50 (Learner Lab)	Dentro del presupuesto

9. Conclusiones

La hipótesis se confirma: la arquitectura de lago de datos en S3 con Spark sobre AWS Glue cumple las metas de tiempo, exactitud y costo con amplio margen.

- **Tiempo total:** 110,6 s para 12 millones de órdenes, frente a la meta propuesta de 2 horas y el límite exigido de 24 horas.
- **Exactitud e integridad:** 100% de coincidencia con el oráculo independiente y conciliación cuadrada, sin pérdidas ni duplicados.
- **Escalabilidad:** el tiempo creció menos que linealmente al multiplicar el volumen por diez.
- **Costo:** unos US$0,10 por corrida y menos de US$25 al mes, incluso con un año de datos almacenados.
- **Trazabilidad y privacidad:** cada decisión guarda regla, versión de catálogo y corrida, y los identificadores personales se seudonimizan con SHA-256 conforme a la Ley 1581 de 2012.

La opción por lotes queda como solución base. La ingesta continua (opción 2) se reserva como extensión si el gobierno necesita decisiones durante el día. Las cifras de costo deben confirmarse con la facturación real del laboratorio.

Referencias

Amazon Web Services [AWS]. (s. f.-a). *Amazon Data Firehose pricing*. Recuperado el 19 de octubre de 2026 de https://aws.amazon.com/firehose/pricing/

Amazon Web Services [AWS]. (s. f.-b). *Amazon EMR pricing*. Recuperado el 19 de octubre de 2026 de https://aws.amazon.com/emr/pricing/

Amazon Web Services [AWS]. (s. f.-c). *Amazon Athena pricing*. Recuperado el 19 de octubre de 2026 de https://aws.amazon.com/athena/pricing/

Casanova, R., Villa-Garzón, F. A., y Branch-Bedoya, J. W. (2025). Architectural patterns for health information systems: A systematic review. *Frontiers in Digital Health, 7*. https://doi.org/10.3389/fdgth.2025.1694839

Cloudburn. (2026). *AWS Glue pricing*. Recuperado el 19 de octubre de 2026 de https://cloudburn.io/blog/aws-glue-pricing

Congreso de Colombia. (2012). *Ley Estatutaria 1581 de 2012, por la cual se dictan disposiciones generales para la protección de datos personales*. Diario Oficial 48.587.

Consultorsalud. (2025, 21 de julio). *Minsalud activa actualización 1.4.4 del Mecanismo Único de Validación FEV-RIPS: avances y desafíos para el sistema de salud*. https://consultorsalud.com/ministerio-salud-fev-rips-optimiza-registros/

Government Accountability Office [GAO]. (2012). *Medicare fraud prevention: CMS has implemented a predictive analytics system, but needs to define measures to determine its effectiveness* (GAO-13-104). https://www.gao.gov/products/gao-13-104

Haghiri, H., Rabiei, R., Hosseini, A., Moghaddasi, H., y Asadi, F. (2019). Notifiable diseases surveillance system with a data architecture approach: A systematic review. *Acta Informatica Medica, 27*(4), 268–277. https://doi.org/10.5455/aim.2019.27.268-277

Herland, M., Khoshgoftaar, T. M., y Bauder, R. A. (2018). Big Data fraud detection using multiple Medicare data sources. *Journal of Big Data, 5*. https://doi.org/10.1186/s40537-018-0138-3

Hevner, A. R., March, S. T., Park, J., y Ram, S. (2004). Design science in information systems research. *MIS Quarterly, 28*(1), 75–105. https://doi.org/10.2307/25148625

Hota, B., Casey, P., McIntyre, A. F., Khan, J., Rab, S., Chopra, A., Lateef, O., y Layden, J. E. (2022). A standard-based citywide health information exchange for public health in response to COVID-19: Development study. *JMIR Public Health and Surveillance, 8*(9), e35973. https://doi.org/10.2196/35973

Kuniyil, L., Hausler, D., Urpi Vidal, S., y Solaz Hernandez, V. (2026, 14 de julio). How Mapfre USA modernized fraud claims with Amazon EMR Serverless. *AWS Architecture Blog*. https://aws.amazon.com/blogs/architecture/how-mapfre-usa-modernized-fraud-claims-with-amazon-emr-serverless/

Ministerio de Salud y Protección Social. (2024). *Resolución 1884 de 2024* (30 de septiembre). Diario Oficial 52.896. https://normograma.supersalud.gov.co/compilacion/docs/resolucion_minsaludps_1884_2024.htm

National Health Authority [NHA]. (2023, 14 de noviembre). *National Health Authority (NHA) organises Accelerator Workshops on ABDM Integration and NHCX Adoption in association with IRDAI* [Comunicado de prensa]. Press Information Bureau. https://abdm.gov.in:8081/uploads/PIB_1976957_1da84d2ad8.pdf

National Health Insurance Administration [NHIA]. (2025, diciembre). *NHIA received the Integrity Award for Excellent Performance in Integrity and Transparency: Ensuring sustainability for the NHI program*. https://www.nhi.gov.tw/en/cp-19444-fefec-8-2.html

National Resource Centre for EHR Standards [NRCES]. (s. f.). *NHCX profiles. FHIR Implementation Guide for ABDM* (v4.0.0). https://nrces.in/ndhm/fhir/r4/4.0.0/hcx-profile.html

Nicholls, S. M., Poplawski, R., Bull, M. J., Underwood, A., Chapman, M., Abu-Dahab, K., Taylor, B., Colquhoun, R. M., Rowe, W. P. M., Jackson, B., Hill, V., O'Toole, Á., Rey, S., Southgate, J., Amato, R., Livett, R., Gonçalves, S., Harrison, E. M., Peacock, S. J., . . . Loman, N. J. (2021). CLIMB-COVID: Continuous integration supporting decentralised sequencing for SARS-CoV-2 genomic surveillance. *Genome Biology, 22*, Artículo 196. https://doi.org/10.1186/s13059-021-02395-y

Nirmitee. (s. f.). *PMJAY claim process 2026: TMS 2.0 vs NHCX for hospital software*. https://nirmitee.io/blog/nhcx-pmjay-tms-ayushman-bharat-claims/

Perdomo, L., Ordinez, L., y Kaminker, S. A. (2026). *A scalable architecture for real-time public health vulnerability assessment* [Artículo de conferencia]. 14th Conference on Cloud Computing, Big Data & Emerging Topics, Universidad Nacional de La Plata. https://sedici.unlp.edu.ar/bitstream/handle/10915/197621/Documento_completo.pdf

Rico Barrera, S. M. (2024, 13 de septiembre). *Transición gradual de los RIPS: nuevos ajustes a la FEV en salud*. Consultorsalud. https://consultorsalud.com/transicion-gradual-rips-fev-salud-pr/

Sachdeva, S., Bhatia, S., Al Harrasi, A., Shah, Y. A., Anwer, K., Philip, A. K., Shah, S. F. A., Khan, A., y Halim, S. A. (2024). Unraveling the role of cloud computing in health care system and biomedical sciences. *PubMed Central* (PMC11004887). https://pmc.ncbi.nlm.nih.gov/articles/PMC11004887/

Schwaber, K., y Sutherland, J. (2020). *La Guía Scrum*. https://scrumguides.org

Shea, E., y Mahapatro, D. (2023, 12 de enero). Building a modern event-driven application for insurance claims processing – Part 1. *AWS for Industries*. https://aws.amazon.com/blogs/industries/building-a-modern-event-driven-application-for-insurance-claims-processing-part-1/

Taramasco, C., y Rimassa, C. (2023). Architecture assessment of the Chilean epidemiological surveillance system for notifiable diseases (EPIVIGILA): Qualitative study. *JMIR Formative Research, 7*, e34387. https://doi.org/10.2196/34387

Upstash. (2026, 9 de octubre). *AWS S3 pricing explained (2026): Storage, requests and data transfer costs*. https://upstash.com/blog/aws-s3-pricing-explained-2026